

PartI: baseline framework

1. Train.py

1.1 Parameter parsing and verification

Parse command-line parameters, check that the number of steps and batch size must be greater than 0; the output directory must be empty to prevent overwriting of previous experiments. If the parameters are invalid, report an error and exit.

1.2 Initialization phase

- Configure the running device (CPU/CUDA), operation precision, thread count, and set the random seed to ensure reproducibility.
- Load the WikiText-2 dataset, read the model configuration file, call `make_model()` to create a GPT model and move it to the corresponding device.
- Create an output folder, initialize the AdamW optimizer.
- Load the training token data to the device, create an independent random number generator specifically for sampling the training window.
- Initialize the training record list to record the start time of the program.

1.3 Main training loop

Each iteration step:

- ① Randomly extract a 257-length token segment from the long text and generate a batch.
- ② Calculate the learning rate
- ③ Zero the gradients
- ④ Perform loss backpropagation
- ⑤ Every 100 steps or the last step: record the training loss, time consumption, and print the log.
- ⑥ If eval-every interval is enabled: run the entire validation set with fp32 precision, save the validation metrics, and accumulate the validation time consumption.

⑦ Return to the loop and execute the next step.

1.4 Training completed, final tasks

- The GPU performs synchronization, calculating the pure training time excluding the validation time consumption.
- Perform a final complete evaluation on the validation set.
- Move the model to the CPU, save checkpoint.pt, only saving the model weights, not the optimizer, and do not support resuming training from a breakpoint.
- Assemble all experimental information: parameter size, training token count, time consumption of each part, loss history, validation metrics, file hash, etc., and write it to metrics.json.
- Print summary results, and the program ends.

2. model.py

2.1 Block: A single-layer Transformer decoding block. It includes LayerNorm, multi-head self-attention (scaled_dot_product_attention, with causal masking enabled, only able to see the previous text), and a 4-fold width MLP with GELU activation; residual connection is after layer normalization.

2.2 GPT Main Model:

- ① Word embeddings tokens + learnable absolute position embeddings pos;
- ② Stack multiple layers of Block, the last layer is LayerNorm;
- ③ The output head performs word table mapping and weight binding (the output head weights reuse the token embedding weights to reduce parameter quantity).

2.3 Forward Logic features(): Input token id, add word embeddings + position embeddings, pass through all Transformer Blocks one by one, and finally normalize; forward(): Output original logits, used for training; predict_log_probs(): Output log-softmax normalized logarithmic probabilities, used in evaluation stage.

ErWeight Initialization: Linear layers and embedding layers use standard deviation 0.02 normal initialization, and biases are zero.

2.4 Task: Text autoregressive prediction, the training objective is cross-entropy loss.

PartII: baseline model performance

1. Deficiencies of the baseline model

1.1 Defect in learning absolute position encoding (baseline position encoding)

①Poor extrapolation ability: Position embeddings are obtained by direct table lookup, and training only involves sequences of (≤ 256)length. If the sequence length during inference is longer than the training context, there are no corresponding embedding parameters, and it is completely impossible to generalize longer texts.

②Inefficient expression of position information: Position encoding is an independent learnable parameter, and it is directly added to the token embedding; the position information is decoupled from the token content and is not integrated into the Q K attention calculation.

③The training step is relatively small, the model has not truly converged yet.

1.2 Deficiencies of GELU activation

Baseline MLP: Linear $\rightarrow$ GELU $\rightarrow$ Linear

①Single-branch structure: Only performs one non-linear mapping, with a lower upper limit of expression ability.

②Simple information flow: There is only one path, and the selectivity for different feature dimensions is weak.

2. Training log

2.1 original recipe

<pre>"bpb": 2.071083638533899, "token_ppl": 79.52826100503707, "nll_nats": 1648039.5698576316, "targets": 376599, "utf8_bytes": 1148007, "seconds": 9.774149099999704</pre>	<pre>"device": "cuda:0", "parameters": 1088256, "precision": "bf16", "train_tokens": 9830400, "preparation_seconds": 12.396927300000243, "train_seconds": 159.78161209999962, "device_name": "NVIDIA GeForce GTX 1050", "peak_allocated_gb": 0.580211712, "peak_reserved_gb": 0.694157312</pre>
---	---

2.2 Recipe(batch_size = 64, steps = 2400)

```

"parameters": 1088256,
"precision": "bf16",
"train_tokens": 32768000,
"preparation_seconds": 12.845767099999648,
"train_seconds": 518.2186223999997,
"validation": {
  "bpb": 1.8345389410010848,
  "peak_allocated_gb": 1.098863616,
  "peak_reserved_gb": 1.24780544
}

```

3. Test log

<pre> "bpb": 2.1012604380869244, "token_ppl": 80.84711881780987, "nll_nats": 1881794.64505621, "targets": 428405, "utf8_bytes": 1292013, "seconds": 11.271238499999527, "protocol": "7506-mp1-wt2-v2", "split": "test", "precision": "fp32", </pre>	<pre> "bpb": 1.8632888298608785, "token_ppl": 49.16069678120102, "nll_nats": 1668677.9414252713, "targets": 428405, "utf8_bytes": 1292013, "seconds": 12.208597799999552, "protocol": "7506-mp1-wt2-v2", "split": "test", </pre>
---	--

original recipe

changed recipe

PartIII: improvement on baseline model

1. Replacing the learnable absolute position encoding with RoPE (Rotary Position Embedding) encoding.

1.1 Principle: Instead of treating position encoding as an additional vector and adding it directly to the token embedding, the Q and K vectors are directly rotated to incorporate the position information into the attention inner product calculation.

However, it might slightly increase the time required for reasoning.

1.2 Advantages: The LLaMA series, Qwen, etc. are all in use.

- ① Stronger preference for relative position induction
- ② Smaller parameter cost
- ③ Better generalization
- ④ Better training stability

1.3 Application:

```
def Student_RoPE(x: torch.Tensor, theta: torch.Tensor) -> torch.Tensor:
    """
    RoPE 旋转位置编码
    x: [B, heads, L, head_dim]
    theta: [L, head_dim/2] 预计算的旋转角度
    """
    B, H, L, D = x.shape
    # 对半拆分, 实部 虚部
    x1 = x[..., ::2]
    x2 = x[..., 1::2]
    # 旋转
    rot_x1 = x1 * torch.cos(theta) - x2 * torch.sin(theta)
    rot_x2 = x1 * torch.sin(theta) + x2 * torch.cos(theta)
    # 合并回原维度
    return torch.stack([rot_x1, rot_x2], dim=-1).flatten(-2)
```

2. Replacing the native GELU with the SwiGLU activation feedforward layer

2.1 Principle of the SwiGLU function:

$$\text{SwiGLU}(\mathbf{x}) = (\mathbf{W}_1 \mathbf{x}) \otimes \text{Swish}(\mathbf{W}_2 \mathbf{x})$$

2.2 Advantages:

- ① Gate control mechanism, with stronger expression ability
- ② Gradient flow is more stable, making training easier to converge
- ③ Better performance with the same parameter quantity; can flexibly scale the hidden dimension
- ④ Better suppression of redundant features

Small model parameters are scarce, and stronger nonlinear expression ability can better fit the text distribution, directly reducing BPB.

2.3 Application:

```
class SwiGLU(nn.Module):
    def __init__(self, width: int, scale: float = 2.75):
        super().__init__()
        hidden = int(width * scale)
        self.w1 = nn.Linear(width, hidden)
        self.w2 = nn.Linear(width, hidden)
        self.w3 = nn.Linear(hidden, width)

    def forward(self, x):
        return self.w3(F.silu(self.w1(x)) * self.w2(x))
```

3. Increasing the learning rate appropriately

A small learning rate is likely to cause the model to get stuck in a local optimal

state, and the convergence speed of the model is relatively slow. On the other hand, a large learning rate is likely to cause the model training to diverge and oscillate. After repeated verification, it was found that when the learning rate peaks at 0.008, the model training can achieve a very good effect. However, when the learning rate is greater than 0.008, the loss and BPB of the model will increase instead. Therefore, 0.008 is chosen.

```
batch = tokens[starts[:,None]+torch.arange(257,device=device)]
learning_rate = .008* min(1.,(step+1)/100) * (.1+.9*.5*(1+math.cos(math.pi*step/args.steps)))
for group in optimizer.param_groups:
```

4. Changing training recipe

Under the original training strategy, the model has not yet truly converged; therefore, modifying the training strategy and increasing the parameter scale can effectively improve model performance. However, this also increases training time and GPU memory usage, and carries a risk of overfitting.

```
💡 p.add_argument('--steps', type=int, default=2000)
p.add_argument('--batch-size', type=int, default=64)
```

PartIV: optimized model

(The performance of student model)

1. Contrastive ablation experiment

①Model architecture has been modified(SwiGLU+RoPE), while other aspects remain unchanged, During training, the BPB value decreased compared to the baseline(BPB = 2.07), However, training time, CPU reasoning time and memory usage is slightly increased.

```

    "step": 1200,
    "bpb": 1.8370295956130693,
    "token_ppl": 48.50011234333982,
    "nll_nats": 1461793.9170786168,
    "targets": 376599,
    "utf8_bytes": 1148007,
    "seconds": 2.046710599999642
  },
  ],
  "intermediate_validation_seconds": 8.195566099999269,
  "process_seconds": 208.69614809999985,
  "torch_version": "2.7.1+cu126",
  "threads": 4,
  "checkpoint_sha256": "d71eb2ebbb6fb6a094bafa293eaa5e918e1c00892e9ddc4965b180a37abd791a",
  "implementation_sha256": "6429acf2523ee2cc6bcd7f7195b2be5226464be0a8f0fdd974697e4cdd26b6716",
  "device": "cuda:0",
  "device_name": "NVIDIA GeForce GTX 1050",
  "peak_allocated_gb": 0.616496128,
  "peak_reserved_gb": 0.70254592

```

②Change the training strategy, while keeping everything else the same.

(learning rate: 0.001 -> 0.006), It can significantly reduce BPB during training.

```

    "bpb": 1.919945834978932,
    "token_ppl": 57.787046039147036,
    "nll_nats": 1527773.5042455848,
    "targets": 376599,
    "utf8_bytes": 1148007,
    "seconds": 1.7299152999999023
  },
  ],
  "history": [],
  "validation_history": [],
  "intermediate_validation_seconds": 0.0,
  "process_seconds": 174.04779429999985,
  "torch_version": "2.7.1+cu126",
  "threads": 4,
  "checkpoint_sha256": "5bf8e2be69ab24fb56df6a5b4dbb602718ba1d40f84974750ec4ca5dd04b65c0",
  "implementation_sha256": "6cce142a31d396f017319543f50d70f30fa6da740729af01c8baed13c6f671",
  "device": "cuda:0",
  "device_name": "NVIDIA GeForce GTX 1050",
  "peak_allocated_gb": 0.580211712,
  "peak_reserved_gb": 0.694157312

```

③Change training recipe, only change batch_size=64 and steps=2000, keep everything else the same as baseline. It can significantly reduce BPB during training, also brings in more parameters, tokens and training memory usage.

```

    "seed": 17,
    "parameters": 1088256,
    "precision": "bf16",
    "train_tokens": 32768000,
    "preparation_seconds": 12.845767099999648,
    "train_seconds": 518.2186223999997,
    "validation": {
      "bpb": 1.8345389410010848,
      "token_ppl": 48.24554375310952,
      "nll_nats": 1459812.0090189765,
      "targets": 376599,
      "utf8_bytes": 1148007,
      "seconds": 2.143767800000205
    }

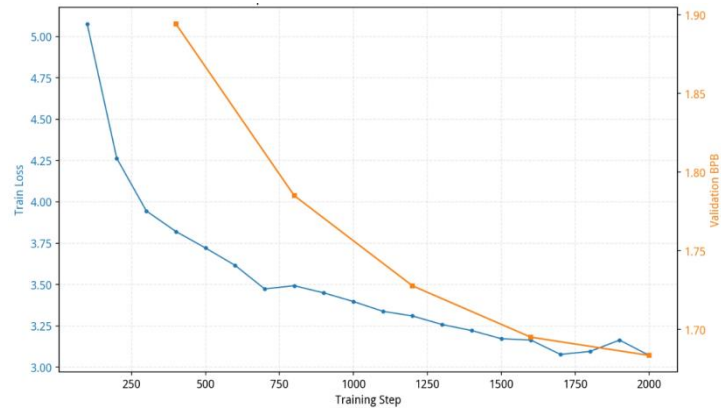
```

```

    "peak_allocated_gb": 1.098863616,
    "peak_reserved_gb": 1.24780544

```

2. Final loss and BPB curves



3. Final training log

```
"train_tokens": 32768000,
"parameters": 1072640,
"preparation_seconds": 11.313437900000281,
"train_seconds": 596.3205366999973,
"bpb": 1.6835886937696813,
"token_ppl": 35.0702066080144,
```

```
"step": 2000,
"bpb": 1.6835886937696813,
"token_ppl": 35.0702066080144,
"nll_nats": 1339695.1890660902,
"targets": 376599,
"utf8_bytes": 1148007,
"seconds": 2.0466882000000624
}
],
"process_seconds": 620.9106668999993,
"torch_version": "2.7.1+cu126",
"threads": 4,
"checkpoint_sha256": "ff91c5af4af76a3364ef8a2d4602ff0315ca184029d7594172ea54678eac0b27",
"implementation_sha256": "c768c05793a356c0f4136079cec112cb0faa318c4985e936d5f734b23675f468",
"device": "cuda:0",
"device_name": "NVIDIA GeForce GTX 1050",
"peak_allocated_gb": 1.172634624,
"peak_reserved_gb": 1.306525696
```

Compared with Contrastive ablation experiment ③, under the same number of tokens, the model of student can achieve a lower BPB.

4. Final test log

```
"bpb": 1.7092029055743063,
"token_ppl": 35.62283980321769,
"nll_nats": 1530685.4955840514,
"targets": 428405,
"utf8_bytes": 1292013,
"seconds": 14.336690699999963,
"protocol": "7506-mp1-wt2-v2",
"split": "test",
"precision": "fp32",
"checkpoint_sha256": "ff91c5af4af76a3364ef8a2d4602ff0315ca184029d7594172ea54678eac0b27",
"implementation_sha256": "c768c05793a356c0f4136079cec112cb0faa318c4985e936d5f734b23675f468",
"evaluator_sha256": "128bcb2dab0be0d427505bddb4671e3ab3a8f78e114be79a689c0f9029af133d",
"tokenizer_sha256": "020d1bc6aa4449c4f352b2e03d0e0fb4f39287f15297705e421b1fa7d817262e",
"device": "cpu",
"device_name": "CPU",
"peak_allocated_gb": 0.0,
"peak_reserved_gb": 0.0
```

5. Quality-computational- cost trade- off between baseline and student model

Metric	Baseline	Student	Comparison Notes
Model Architecture	Learnable absolute positional encoding + GELU-MLP	RoPE +SwiGLU	Two structural architectural improvements
Training recipe	Batch_size = 32 Steps = 1200	Batch_size = 64 Steps = 2000	Sufficiently study and reduction of noise,but bring more tokens, time costs and memory costs
Learning rate	0.001	0.008	Provides faster convergence speed and learning efficiency
Parameter Count	1,088,256	1,072,640	Reduced by 15,616 (attributed to RoPE)
Total Training Tokens	9,830,400	32,768,000	Caused by the change of Training recipe
Final Training Loss	4.317	3.071	Decreased by 1.638
Test-set BPB	2.101	1.709	Decreased by 0.392
Validation-set BPB	2.071	1.683	Decreased by 0.388
Test-set Token PPL	79.52	35.07	Decreased by 44.45
Test Inference Latency (CPU, FP32)	11.27 s	14.34 s	More reasoning time but meet requirements
Peak GPU Memory Usage	0.580 GB	1.173 GB	More memory usage but meet requirement: ≤ 4 GB

PartV: conclusion

Despite the increase in training time, the CPU inference time and the memory usage. based on the baseline model, improvements were made in both the structure and the training strategy. the following results were achieved:

1. **Significant performance improvement:** The BPB of the test set decreased from 2.101 to 1.709, a reduction of 0.388, the token PPL decreased from 79.52 to 35.70.
2. **The number of parameters is even less:** student's model has less parameters(1,072,640) than baseline.
3. **All resource constraints were met:** Although the CPU inference time has increasedafter the improvement, CPU inference time was $14.34s < 56.53s$ (the upper limit), VRAM was $1.173GB < 4GB$, constraints were met.
4. **No sign of overfitting:** The BPB remained monotonically decreasing throughout the process, and the difference of BPB between val and test was very small ($\Delta \approx 0.03$).
5. **According to final training log and Contrastive ablation experiment ③ :** student's model can still achieve lower BPB while more tokens(3 times the baseline) was involved.